LAB12 車牌辨識專題實驗報告

614530036 邱世宇 資管碩一 jasonchiu0916@gmail.com

一、實驗名稱

車牌辨識專題：OpenALPR 與 OpenCV 車牌辨識流程實作

二、實驗目的

- 建構一個簡易的車牌辨識系統，理解 ALPR（Automatic License Plate Recognition，自動車牌辨識）的基本流程。
- 學習在 Raspberry Pi 上安裝影像處理與辨識相關環境，包括 OpenALPR、OpenCV、Leptonica 與 Tesseract OCR。
- 理解 OpenALPR 可分析圖片或影片串流，並輸出車牌文字與信心分數。
- 理解 plate_recog 專案的手寫辨識流程，包含灰階、雙邊濾波、Canny 邊緣、輪廓偵測、透視校正、Otsu 二值化、形態學處理與模板比對。
- 熟悉透過 SSH / SFTP 對 Raspberry Pi 進行遠端操作、檔案傳輸、Python 虛擬環境建置與程式測試。
- 透過執行結果、截圖、時間量測與空間量測，驗證車牌辨識程式是否能正確運作。

三、實驗步驟

3.1 實驗環境與材料

類別	內容
硬體	Raspberry Pi、已安裝 Raspberry Pi OS 的 SD card、Wi-Fi 網路環境
本機端	Linux 電腦或可執行 SSH / SFTP 的開發環境
系統工具	ssh、sftp、git、cmake、make、gcc/g++、Python virtual environment
辨識套件	OpenALPR、OpenCV、Leptonica、Tesseract OCR

3.2 透過 SSH 連線並更新 Raspberry Pi

先以 SSH 遠端登入 Raspberry Pi，接著更新套件清單與系統套件，確保後續編譯與安裝流程使用最新依賴。

```
$ sudo apt update
$ sudo apt upgrade
```

3.3 安裝基礎編譯與影像處理相關套件

安裝 autoconf、automake、libtool、cmake、OpenCV 編譯依賴與數值運算相關函式庫，作為後續建置 Leptonica、Tesseract 與 OpenALPR 的基礎。

```
$ sudo apt install autoconf automake libtool -y
$ sudo apt install build-essential cmake git libgtk2.0-dev pkg-config libavcodec-dev libavformat-dev libswscale-dev -y
$ sudo apt install libatlas-base-dev gfortran -y
```

3.4 視需求增加 swap 空間

若 Raspberry Pi 記憶體不足導致編譯失敗，可暫時建立 swapfile 增加可用記憶體；若系統資源足夠，此步驟可略過。

```
$ sudo fallocate -l 4G /swapfile
$ sudo chmod 600 /swapfile
$ sudo mkswap /swapfile
$ sudo swapon /swapfile
$ sudo swapon --show
```

3.5 安裝 Leptonica

Leptonica 是常用於影像處理與影像分析的函式庫，提供縮放、二值化、形態學與影像轉換等功能，並常作為 OCR 系統的底層依賴。

```
$ cd
$ git clone https://github.com/DanBloomberg/leptonica.git
$ cd leptonica/
$ git checkout v1.71
$ chmod 777 configure
$ ./configure
$ make -j2
$ sudo make install
```

3.6 安裝 Tesseract OCR

Tesseract 是 OCR（Optical Character Recognition，光學字元辨識）引擎，可將圖片中的字元轉換為文字；在車牌辨識情境中，OCR 可協助辨識車牌上的英文字母與數字。

```
$ cd
$ git clone https://github.com/tesseract-ocr/tesseract.git
$ cd tesseract
$ git checkout 3.04.01
$ ./autogen.sh
$ ./configure --enable-debug
$ make -j2
$ sudo make install
```

3.7 安裝 OpenCV

OpenCV 提供影像讀取、灰階轉換、濾波、邊緣偵測、輪廓分析與模板比對等功能，是本實驗手寫車牌辨識流程的核心工具。

```
$ sudo apt install libopencv-dev
```

3.8 安裝 OpenALPR

OpenALPR 是開源自動車牌辨識工具，可針對圖片或影像串流偵測車牌位置並輸出候選車牌號碼與 confidence（信心分數）。

```
$ cd
$ git clone https://github.com/openalpr/openalpr.git
$ cd openalpr
$ sudo apt install libcurl4-openssl-dev liblog4cplus-dev -y
$ cd src
```

```
$ mkdir build
$ cd build
$ cmake ..
$ make -j2
$ sudo make install
$ sudo ldconfig
```

3.9 測試 OpenALPR

使用測試圖片執行 alpr 指令，確認 OpenALPR 能輸出車牌候選結果與信心分數；若可正常輸出 plate0、plate1 等結果，代表 OpenALPR 安裝與基本辨識流程成功。

```
$ alpr plate.jpeg
```

3.10 使用 SFTP 傳輸 plate_recog 專案

若本機可透過 SSH 連線到 Raspberry Pi，則可使用 SFTP 將 plate_recog 專案資料夾傳送到 Raspberry Pi。

```
$ sftp pi@192.168.xxx.xxx
sftp> put -r plate_recog/
```

3.11 建立 Python 虛擬環境並安裝套件

在 plate_recog 專案內建立 virtual environment（虛擬環境），可隔離 Python 套件版本，避免影響系統原本的 Python 環境。

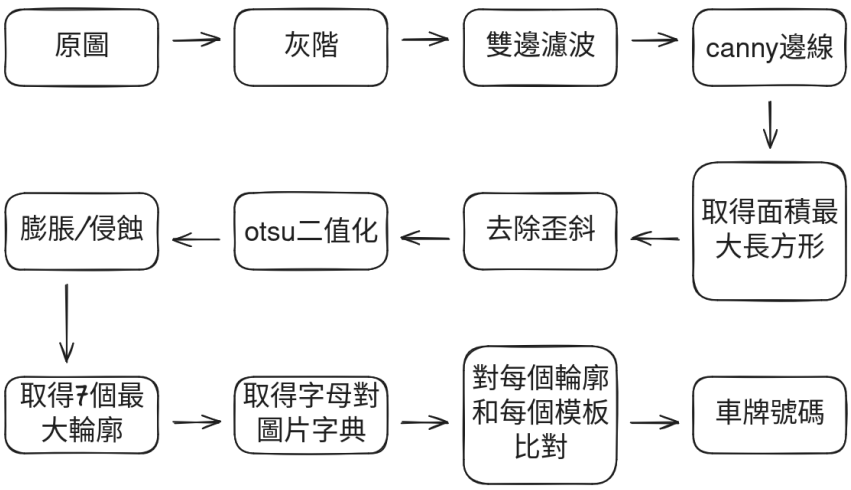
```
$ cd plate_recog
$ python -m venv env
$ source env/bin/activate
$ pip install opencv-python
```

3.12 執行 plate_recog.py

執行手寫車牌辨識程式，觀察終端機輸出的 best match、value 與 plate number，確認程式能對車牌字元進行模板比對並輸出辨識結果。

```
$ python plate_recog.py
# 預期輸出包含：
# best match: A, value: ...
# plate number: AAA4S85
```

3.13 plate_recog 核心辨識流程



處理階段	主要功能	OpenCV / 方法
灰階轉換	降低影像維度，讓後續邊緣與二值化處理更穩定	cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
雙邊濾波	保留邊緣並降低雜訊，避免車牌邊界被過度模糊	cv2.bilateralFilter
Canny 邊緣	抓出影像中亮度變化大的邊界，作為輪廓偵測基礎	cv2.Canny
最大長方形輪廓	從輪廓中尋找最可能為車牌的四邊形區域	cv2.findContours、cv2.approxPolyDP
去除歪斜	透過透視轉換把傾斜車牌轉成正面長方形	cv2.getPerspectiveTransform、cv2.warpPerspective
Otsu 二值化	將灰階影像轉成黑白影像，凸顯字元形狀	cv2.threshold + cv2.THRESH_OTSU
侵蝕/膨脹	消除小雜訊並連接字元區塊	cv2.erode、cv2.dilate
取得 7 個最大輪廓	擷取車牌上的 7 個主要字元區域	cv2.connectedComponentsWithStats
模板比對	將每個字元輪廓與字母/數字模板比較，取最高相似度結果	cv2.matchTemplate

四、作業 demo

4.1 OpenALPR 執行結果

```
問題    輸出    偵錯主控台    終端機    連接埠

(env) jason@JasonPi:~/lab12/img $ alpr plate1.jpg
No license plates found.
(env) jason@JasonPi:~/lab12/img $ alpr plate2.jpg
plate0: 10 results
    - MUY5686     confidence: 89.9899
    - HUY5686     confidence: 83.9382
    - MUY568      confidence: 82.9243
    - NUY5686     confidence: 82.8943
    - MUY56B6     confidence: 82.5521
    - UY5686      confidence: 81.1681
    - MUYS686     confidence: 80.0527
    - KUY5686     confidence: 79.9711
    - BUY5686     confidence: 79.9286
    - MDY5686     confidence: 79.178
```

4.2 plate_recog 截圖

4.2.1 終端機輸出

```
(env) jason@JasonPi:~/lab12 $ python plate_recog.py
best match: A, value: 0.8527087569236755
best match: A, value: 0.8708062171936035
best match: A, value: 0.8740372061729431
best match: 4, value: 0.3543696701526642
best match: S, value: 0.47137004137039185
best match: 8, value: 0.7070448994636536
best match: 5, value: 0.7479891180992126
plate number: AAA4S85
Saved result/plate_boarder_img.jpg
```

4.2.2 框選圖



4.2.3 以程式在圖片上印出方框四個點的座標，並貼上結果截圖。



4.2.4 紀錄不含產圖的執行時間，以及 env、程式碼與模板的總空間大小。

```
(env) jason@JasonPi:~/lab12 $ python plate_recog.py
best match: A, value: 0.8527087569236755
best match: A, value: 0.8708062171936035
best match: A, value: 0.8740372061729431
best match: 4, value: 0.3543696701526642
best match: S, value: 0.47137004137039185
best match: 8, value: 0.7070448994636536
best match: 5, value: 0.7479891180992126
plate number: AAA4S85
P1: x=424, y=16
P2: x=441, y=176
P3: x=82, y=300
P4: x=41, y=91
Saved result/plate_boarder_img.jpg
=====================================
Program runtime: 0.1815 seconds
=====================================
```

```
(env) jason@JasonPi:~/lab12 $ du -sh ~/env
200M    /home/jason/env
(env) jason@JasonPi:~/lab12 $ du -sh ~/lab12/plate_recog.py
8.0K    /home/jason/lab12/plate_recog.py
(env) jason@JasonPi:~/lab12 $ du -sh ~/lab12/template/
148K    /home/jason/lab12/template/
```

200M + 8.0K + 148K ≈ 200.16M

4.3 列出三個可能造成問題的點

問題一：固定假設車牌一定有 7 個字元

程式中寫死 num_of_digits = 7，代表系統永遠只抓取 7 個最大連通元件。若車牌字元數不是 7 個，或螺絲、雜訊被誤判成字元，就會造成辨識錯誤。

問題二：只使用最大四邊形輪廓判斷車牌位置

程式透過 approxPolyDP() 找出 4 個角點來當作車牌框，但若圖片中有其他矩形物件，例如車燈、保險桿、裝飾框，可能會誤抓成車牌。這會導致後續透視校正與字元切割全部錯誤。

問題三：模板比對容易受到字體、大小與角度影響

程式使用 cv2.matchTemplate() 進行字元比對，這種方法對字型、粗細、旋轉角度與影像品質很敏感。若車牌字元與模板資料夾內的字元差異太大，就容易把相似字誤判，例如 6 與 G、8 與 B、0 與 O。

4.4 從問題中選一項進行改善



原本　　**改善**

針對問題三：模板比對容易受到字體、大小與角度影響，主要改善就是新增這 3 個函式：

def normalize_char_image

功能：把字元圖片和模板都統一成 灰階、固定大小、二值化，降低亮度、尺寸不同造成的誤差。

```
def normalize_char_image(image, size=(18, 36)):
    # 統一轉成灰階
    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 統一大小
    image = cv2.resize(image, size)

    # 統一二值化，降低亮度差異影響
    _, image = cv2.threshold(
        image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    return image
```

def rotate_image(image, angle):

功能：讓模板可以產生不同旋轉角度，例如 -10、-5、0、5、10 度，提高對字元傾斜的容錯能力。

```
def rotate_image(image, angle):
    h, w = image.shape[:2]
    center = (w // 2, h // 2)

    matrix = cv2.getRotationMatrix2D(center, angle, 1.0)

    rotated = cv2.warpAffine(
        image,
        matrix,
        (w, h),
        flags=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_CONSTANT,
        borderValue=0
    )

    return rotated
```

def get_plate_str_improved(images, templates, logs=False):

功能：取代原本的 get_plate_str()，加入 角度容錯、大小容錯、車牌格式限制。

其中格式限制是：前 3 碼只比對英文字母，後 4 碼只比對數字，所以可以避免第 4 碼 6 被誤判成 K。

```
def get_plate_str_improved(images, templates, logs=False):
    plate_str = ""

    angles = [-10, -5, 0, 5, 10]
    scales = [0.9, 1.0, 1.1]
    target_size = (18, 36)

    for idx, parts in enumerate(images):
        best_match = None
        best_match_value = -1

        part_img = normalize_char_image(parts, target_size)
```

```
    for key, template in templates.items():

        # 前 3 碼只允許英文字母
        if idx < 3 and not key.isalpha():
            continue

        # 後 4 碼只允許數字
        if idx >= 3 and not key.isdigit():
            continue

        template_img = normalize_char_image(template, target_size)

        for scale in scales:
            scaled_w = int(target_size[0] * scale)
            scaled_h = int(target_size[1] * scale)

            scaled_template = cv2.resize(template_img, (scaled_w, scaled_h))
            scaled_template = cv2.resize(scaled_template, target_size)

            for angle in angles:
                rotated_template = rotate_image(scaled_template, angle)

                result = cv2.matchTemplate(
                    part_img,
                    rotated_template,
                    cv2.TM_CCOEFF_NORMED
                )

                _, max_val, _, _ = cv2.minMaxLoc(result)

                if max_val > best_match_value:
                    best_match = key
                    best_match_value = max_val

    plate_str += best_match

    if logs:
        print(f"char {idx+1}: best match: {best_match}, value: {best_match_value}")

return plate_str
```

主程式改成呼叫：

```
plate_str = get_plate_str_improved(component_images, templates, logs=True)
```

4.5 心得

這次實作車牌辨識讓我更了解 OpenCV 影像處理的流程，不是只有把圖片丟進程式就能得到正確結果，而是需要經過灰階化、濾波、邊緣偵測、找輪廓、透視校正、字元切割和模板比對等步驟。實作過

程中我發現，車牌辨識的準確度很容易受到光線、角度、字體大小和背景雜訊影響，例如原本程式把數字 6 誤判成英文字母 K，代表單純使用模板比對其實不夠穩定。

為了改善這個問題，我新增了字元標準化、模板旋轉，以及改良版模板比對函式，讓程式可以先把字元和模板統一成相同大小與二值化格式，再用不同角度和比例去比對。另外也加入車牌格式限制，讓前三碼只比對英文字母，後四碼只比對數字，降低相似字元誤判的機率。這次實作讓我學到，影像辨識不只是寫出可以跑的程式，更重要的是觀察錯誤結果，找出問題發生在哪一個處理階段，再針對那個部分做改善。

plate_recog.py

```
import os
import cv2
import numpy as np
import time
import argparse



def get_largest_rect_contour(image):
    contours, _ = cv2.findContours(
        image, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
    contours = sorted(contours, key=cv2.contourArea, reverse=True)[:10]
    rect_contour = None
    for c in contours:
        p = cv2.arcLength(c, True)
        approx = cv2.approxPolyDP(c, 0.02 * p, True)
        if len(approx) == 4:
            rect_contour = approx
            break

    if rect_contour is None:
        raise Exception("No rect found")
    return rect_contour
```

```
def get_ordered_rect_points(contour):
    # the order is, bottom-left, bottom-right, top-right, top-left
    rect = np.zeros((4, 2), dtype="float32")
    s = contour.sum(axis=2)
    rect[0] = contour[np.argmin(s)]
    rect[2] = contour[np.argmax(s)]
    diff = np.diff(contour, axis=2)
    rect[1] = contour[np.argmin(diff)]
    rect[3] = contour[np.argmax(diff)]
    return rect



def deskew(src_img, base_rect, skewed_rect):
    M = cv2.getPerspectiveTransform(skewed_rect, base_rect)
    dest_img = cv2.warpPerspective(
        src_img, M, (int(base_rect[2][0]), int(base_rect[2][1])))
    return dest_img



def get_n_largest_components(image, n):
    num_labels, labels, stats, _ = cv2.connectedComponentsWithStats(
        image, connectivity=8)

    areas = stats[:, cv2.CC_STAT_AREA]
    sorted_areas_indices = np.argsort(-areas)

    component_images = []
    component_image_x = []

    for i in range(1, n + 1):
```

```
        largest_component = sorted_areas_indices[i]

        x = stats[largest_component, cv2.CC_STAT_LEFT]
        y = stats[largest_component, cv2.CC_STAT_TOP]
        width = stats[largest_component, cv2.CC_STAT_WIDTH]
        height = stats[largest_component, cv2.CC_STAT_HEIGHT]

        component_image = np.zeros_like(image)
        component_image[labels == largest_component] = 255
        component_image = component_image[y:y+height, x:x+width]
        component_images.append(component_image)
        component_image_x.append(x)

    # Sort the components by x position
    component_images = [x for _, x in sorted(
        zip(component_image_x, component_images))]

    return component_images


def get_templates(dir_path):
    templates = {}
    for file in os.listdir(dir_path):
        if file.endswith(".jpg"):
            template_image = cv2.imread(os.path.join(dir_path, file), 0)
            filename = os.path.splitext(file)[0]
            templates[filename] = template_image
    # key: filename, value, template_image
    return templates
```

```
def draw_result(img, rect_contour, plate_str, output_path="result/plate_boarder_img.jpg"):
    res_img = img.copy()
    points = rect_contour.reshape(4, 2)
    h, w = res_img.shape[:2]

    # 1. 先畫綠色框
    cv2.drawContours(res_img, [rect_contour], -1, (0, 255, 0), 2)

    # 2. 再畫四個座標點與文字
    for i, (x, y) in enumerate(points):
        x = int(x)
        y = int(y)

        cv2.circle(res_img, (x, y), 4, (0, 0, 255), -1)

        label = f"P{i+1}:({x},{y})"

        text_x = x + 5
        text_y = y - 5

        if text_x > w - 120:
            text_x = x - 110

        if text_y < 15:
            text_y = y + 18

        if text_y > h - 10:
            text_y = y - 10

        cv2.putText(res_img, label,
                    (text_x, text_y),
```

```
                cv2.FONT_HERSHEY_SIMPLEX,
                0.4, (0, 0, 255), 1)

        print(f"P{i+1}: x={x}, y={y}")

    # 3. 畫車牌辨識結果
    cv2.putText(res_img, plate_str, (30, 40),
                cv2.FONT_HERSHEY_SIMPLEX,
                0.8, (255, 0, 0), 2)

    # 4. 存到 result 資料夾
    os.makedirs(os.path.dirname(output_path), exist_ok=True)
    cv2.imwrite(output_path, res_img)

    print(f'Saved {output_path}')

def print_runtime(start_time):
    end_time = time.time()
    elapsed_time = end_time - start_time

    print("=" * 40)
    print(f'Program runtime: {elapsed_time:.4f} seconds')
    print("=" * 40)

def normalize_char_image(image, size=(18, 36)):
    # 統一轉成灰階
    if len(image.shape) == 3:
        image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # 統一大小
```

```
    image = cv2.resize(image, size)

    # 統一二值化，降低亮度差異影響
    _, image = cv2.threshold(
        image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
    )

    return image

def rotate_image(image, angle):
    h, w = image.shape[:2]
    center = (w // 2, h // 2)

    matrix = cv2.getRotationMatrix2D(center, angle, 1.0)

    rotated = cv2.warpAffine(
        image,
        matrix,
        (w, h),
        flags=cv2.INTER_LINEAR,
        borderMode=cv2.BORDER_CONSTANT,
        borderValue=0
    )

    return rotated

def get_plate_str_improved(images, templates, logs=False):
    plate_str = ""

    angles = [-10, -5, 0, 5, 10]
    scales = [0.9, 1.0, 1.1]
```

```
target_size = (18, 36)

for idx, parts in enumerate(images):
    best_match = None
    best_match_value = -1

    part_img = normalize_char_image(parts, target_size)

    for key, template in templates.items():

        # 前 3 碼只允許英文字母
        if idx < 3 and not key.isalpha():
            continue

        # 後 4 碼只允許數字
        if idx >= 3 and not key.isdigit():
            continue

        template_img = normalize_char_image(template, target_size)

        for scale in scales:
            scaled_w = int(target_size[0] * scale)
            scaled_h = int(target_size[1] * scale)

            scaled_template = cv2.resize(template_img, (scaled_w, scaled_h))
            scaled_template = cv2.resize(scaled_template, target_size)

            for angle in angles:
                rotated_template = rotate_image(scaled_template, angle)

                result = cv2.matchTemplate(
```

```
                part_img,
                rotated_template,
                cv2.TM_CCOEFF_NORMED
            )

            _, max_val, _, _ = cv2.minMaxLoc(result)

            if max_val > best_match_value:
                best_match = key
                best_match_value = max_val

        plate_str += best_match

        if logs:
            print(f"char {idx+1}: best match: {best_match}, value: {best_match_value}")

    return plate_str

def main():

    start_time = time.time()

    parser = argparse.ArgumentParser()

    parser.add_argument("--image_path", type=str,
                        help="path to image file", default="img/plate1.jpg")
    parser.add_argument("--template_path", type=str,
                        help="path to template folder", default="template")
    parser.add_argument("--show_image", type=bool,
                        help="show image, Value: True or False", default=False)
```

```
    img = cv2.imread(parser.parse_args().image_path)

    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    filtered = cv2.bilateralFilter(gray, 11, 17, 17)

    edge = cv2.Canny(filtered, 30, 200)

    rect_contour = get_largest_rect_contour(edge)

    ordered_rect_points = get_ordered_rect_points(rect_contour)

    plate_size_match = (int(320), int(150))
    base_plate_points = np.array([[0, 0], [plate_size_match[0], 0], [plate_size_match[0], plate_size_match[1]],
[
        0, plate_size_match[1]]], dtype="float32")

    plate = deskew(gray, base_plate_points, ordered_rect_points)

    _, plate = cv2.threshold(
        plate, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    plate = cv2.bitwise_not(plate)

    kernel = np.ones((3, 3), np.uint8)
    plate = cv2.erode(plate, kernel, iterations=1)
    plate = cv2.dilate(plate, kernel, iterations=1)

    num_of_digits = 7

    component_images = get_n_largest_components(plate, num_of_digits)
```

```
    template_size = (18, 36)

    for i in range(num_of_digits):
        component_images[i] = cv2.resize(component_images[i], template_size)

    templates = get_templates(parser.parse_args().template_path)

    plate_str = get_plate_str_improved(component_images, templates, logs=True)
    print(f"plate number: {plate_str}")

    #畫框和座標
    draw_result(img, rect_contour, plate_str)

    #結束
    print_runtime(start_time)

    # cv2.waitKey(0)

if __name__ == "__main__":
    main()
```